

**RANCANG BANGUN APLIKASI PENGELOLAAN KEUANGAN PRIBADI
DAN INVESTASI BERBASIS WEB FYNACCHIO: PERSONAL FINANCE
& INVESTMENT TRACKER**



Dosen Pengampu :

Hendry Gunawan, S.Kom., MM

Disusun Oleh:

Titania Pryta Vexia 20250801303

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026**

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
DAFTAR TABEL.....	5
BAB I.....	6
1.1 Latar Belakang.....	6
1.2 Deskripsi Aplikasi.....	7
1.3 Peran Individu.....	7
BAB II.....	8
2.1 Pembagian Fungsi Modular.....	8
2.2 Skema Database SQL	20
BAB III	30
3.1 Tangkapan Layar (Screenshot) UI.....	30
3.2 Penanganan Error (Exception).....	40
BAB IV	42
4.1 Kesimpulan	42
4.2 Tautan Wajib.....	42

DAFTAR GAMBAR

Gambar 2.1 Arsitektur aplikasi web FynaCchio	10
Gambar 2.2 Struktur folder proyek FynaCchio	10
Gambar 2.3 ERD database fynacchio	21
Gambar 3.1 Halaman registrasi pengguna baru	30
Gambar 3.2 Menu login FynaCchio.....	30
Gambar 3.3 Halaman pengeditan profil dan avatar	31
Gambar 3.4 Dashboard berisi ringkasan keuangan dan saldo akun.....	31
Gambar 3.5 Grafik, kalender, dan riwayat transaksi pada dashboard	32
Gambar 3.6 Halaman daftar transaksi.....	33
Gambar 3.7 Form penambahan transaksi.....	33
Gambar 3.8 Hasil penambahan transaksi.....	34
Gambar 3.9 Form pengeditan transaksi	34
Gambar 3.10 Hasil pengeditan transaksi	35
Gambar 3.11 Pemilihan transaksi yang akan dihapus	35
Gambar 3.12 Konfirmasi penghapusan transaksi	36
Gambar 3.13 Hasil penghapusan transaksi	36
Gambar 3.14 Form transfer dana antarakun	37
Gambar 3.15 Ringkasan portfolio, aset, dan target tabungan	37
Gambar 3.16 Riwayat transaksi investasi	38
Gambar 3.17 Form pembelian atau penjualan aset	38
Gambar 3.18 Halaman anggaran dan realisasi pengeluaran	39
Gambar 3.19 Halaman laporan buku besar bulanan	39
Gambar 3.20 Hasil ekspor laporan dalam format PDF	40
Gambar 3.21 Penolakan registrasi karena nama pengguna telah digunakan	40

Gambar 3.22 Peringatan penghapusan tanpa memilih transaksi	41
Gambar 3.23 Penolakan transfer karena saldo akun sumber tidak mencukupi	41

DAFTAR TABEL

Tabel 1.1 Fitur aplikasi FynaCchio.....	7
Tabel 2.1 Fungsi pada modul <code>database.py</code>	11
Tabel 2.2 Fungsi pada modul <code>analytics.py</code>	12
Tabel 2.3 Fungsi pada modul <code>utils.py</code>	12
Tabel 2.4 Fungsi autentikasi dan profil pada modul <code>app.py</code>	13
Tabel 2.5 Fungsi dashboard dan pengolahan data pada modul <code>app.py</code>	14
Tabel 2.6 Fungsi transaksi dan akun pada modul <code>app.py</code>	16
Tabel 2.7 Fungsi portfolio dan investasi pada modul <code>app.py</code>	17
Tabel 2.8 Fungsi anggaran dan target tabungan pada modul <code>app.py</code>	18
Tabel 2.9 Fungsi laporan pada modul <code>app.py</code>	19
Tabel 2.10 Rancangan struktur tabel database FynaCchio	22
Tabel 2.1 Relasi foreign key database FynaCchio	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan keuangan pribadi sering dilakukan secara terpisah melalui catatan manual, aplikasi perbankan, dompet digital, dan platform investasi. Kondisi tersebut menyulitkan pengguna ketika ingin melihat gambaran menyeluruh mengenai pemasukan, pengeluaran, saldo aktif, anggaran, target tabungan, serta perkembangan aset dalam satu tempat. Akibatnya, keputusan finansial dapat dibuat berdasarkan informasi yang tidak lengkap atau terlambat. Selain itu, aktivitas investasi dan target tabungan sering tercampur dengan pencatatan pengeluaran rutin, sehingga sulit dibedakan mana dana yang benar-benar terpakai untuk kebutuhan konsumtif dan mana yang dialokasikan sebagai aset produktif atau simpanan masa depan.

Permasalahan lain muncul pada konsistensi pencatatan. Transaksi harian dapat terlewat, kategori pengeluaran tidak seragam, dan perpindahan dana antar akun berpotensi dicatat sebagai pemasukan atau pengeluaran baru. Tanpa pengolahan data yang baik, pengguna juga sulit mengetahui rasio tabungan, kondisi surplus atau defisit, sisa anggaran, dan pola pengeluaran berdasarkan hari, bulan, atau tahun.

FynaCchio dikembangkan sebagai aplikasi *personal finance & investment tracker* berbasis web untuk mengatasi masalah tersebut. Aplikasi menyimpan data secara permanen pada MySQL dan menyajikannya melalui antarmuka *browser*. Pengguna dapat mencatat arus kas, mengelola rekening, memindahkan dana, mencatat pembelian atau penjualan aset, mengatur target keuangan, membuat anggaran, serta membaca laporan buku besar.

Pemilihan platform web juga mendukung syarat antarmuka visual pada proyek akhir. Aplikasi tidak bergantung pada menu teks terminal. Flask dan Werkzeug menjalankan server yang terus menunggu *request* dari *browser* sampai proses server dihentikan. Karena itu, sifat aplikasi yang terus hidup dicapai melalui event/request loop milik web server, bukan melalui menu CLI dengan `while True`.

1.2 Deskripsi Aplikasi

FynaCchio menyediakan alur autentikasi, *dashboard* analitik, transaksi, investasi/portofolio, anggaran, tabungan (*goals*), laporan arus kas, dan pengelolaan profil. Seluruh *route* yang memproses data pengguna dibatasi oleh `@login_required` dan *query* selalu menyertakan `user_id` agar data antarpengguna terisolasi.

Modul Fitur	Kemampuan
Autentikasi & profil	Registrasi, <i>login</i> , <i>logout</i> , edit profil (meliputi nama tampilan, <i>username</i> , <i>avatar</i>), dan <i>hash password</i> .
<i>Dashboard</i>	Ringkasan keuangan berupa skor kesehatan finansial, rasio tabungan, kas/saldo cair, estimasi <i>net worth</i> , saldo per akun, kalender transaksi bulanan dengan detail harian, serta grafik distribusi pengeluaran, alokasi investasi, dan tren pemasukan, pengeluaran, atau pembelian investasi.
Transaksi	Pencatatan pemasukan dan pengeluaran, pencarian dan filter transaksi, sorting tabel tiga <i>state</i> (<i>ascending</i> , <i>descending</i> , <i>default</i>), edit dan hapus transaksi terpilih, serta transfer dana antarakun.
Investasi & <i>goals</i>	Pengelolaan platform <i>investasi</i> , pencatatan pembelian dan penjualan aset, ringkasan <i>holdings</i> aktif beserta kuantitas aset, dan pengelolaan target tabungan (<i>goals</i>).
Anggaran	Anggaran kategori per periode, realisasi, persentase <i>burn rate</i> , serta pembaruan nominal.
Laporan	Buku besar akuntansi yang menghitung debit, kredit, <i>net cashflow</i> , dan saldo berjalan per bulan, serta fitur ekspor laporan arus kas dalam format PDF bulanan.

Tabel 1.1 Fitur aplikasi *FynaCchio*

1.3 Peran Individu

Proyek ini dikerjakan secara individu oleh Titania Pryta Vexia dengan NIM 20250801303, mulai dari tahap analisis kebutuhan dan pemilihan studi kasus, perancangan antarmuka pengguna (UI/UX), penyusunan skema *database* MySQL, hingga penulisan seluruh kode program yang mencakup *backend* Flask, *query* SQL, serta *frontend* HTML, CSS, JavaScript, dan Bootstrap. Seluruh proses pengujian fungsional aplikasi, penyusunan dokumen laporan, pembuatan naskah, serta perekaman video demonstrasi juga dikerjakan secara individu.

BAB II

ARSITEKTUR SISTEM & DATABASE

2.1 Pembagian Fungsi Modular

Arsitektur Aplikasi

FynaCchio dibangun sebagai aplikasi web dengan pola *client-server*, bukan sebagai aplikasi *Command Line Interface* atau CLI. *Browser* berfungsi menampilkan antarmuka yang dibentuk menggunakan HTML, CSS, JavaScript, Bootstrap, dan Chart.js. Ketika pengguna menekan tombol, mengirimkan formulir, atau berpindah menu, *browser* akan mengirimkan HTTP *request* kepada aplikasi Flask.

Request tersebut diterima oleh *routing* Flask dan diarahkan ke fungsi yang sesuai dengan URL yang diakses. Pada kode FynaCchio terdapat 25 *route*, antara lain */transactions*, */portfolios*, */budgets*, */reports*, dan */export/pdf*. Setelah *request* diterima, Python menjalankan validasi *input*, autentikasi, perhitungan finansial, serta operasi *create*, *read*, *update*, dan *delete*. Jika proses membutuhkan data, Flask akan menjalankan *query* pada *database* MySQL *fynacchio*.

Data yang diperoleh dari MySQL kemudian diolah oleh Python dan dimasukkan ke dalam *template* menggunakan Jinja2 melalui fungsi `render_template_string()`. Hasil *template* dikirim kembali ke *browser* dalam bentuk halaman HTML. Untuk menampilkan data visual, Chart.js digunakan pada sisi *browser* untuk membentuk grafik distribusi pengeluaran, alokasi investasi, tren arus kas, dan realisasi anggaran berdasarkan data yang telah disiapkan oleh Flask.

FynaCchio tidak menggunakan `while True` secara manual. Flask menjalankan *server* melalui Werkzeug dengan perintah `app.run(debug=True, port=5000)`. *Server* tersebut terus menerima dan memproses *request* dari *browser* sampai proses *server* dihentikan. Dengan demikian, aplikasi tetap berjalan setelah satu operasi selesai. Perpindahan antarmenu juga dilakukan melalui *routing* URL, bukan melalui menu teks berulang seperti pada aplikasi CLI.

Autentikasi pengguna memanfaatkan Flask *session*. Setelah proses *login* berhasil, ID pengguna disimpan pada `session['user_id']`. *Route* yang memerlukan status *login* dilindungi menggunakan *decorator* `@login_required`. *Decorator* tersebut memeriksa *session* sekaligus memastikan bahwa ID pengguna masih tersedia pada tabel *users*. Jika

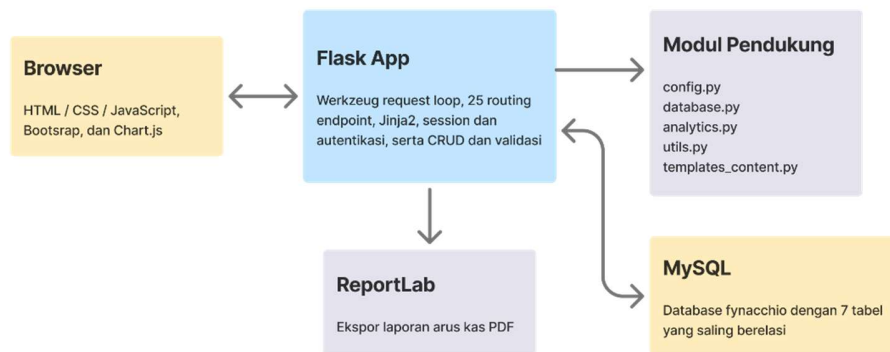
session tidak ditemukan atau sudah tidak valid, pengguna akan diarahkan kembali ke halaman *login*.

FynaCchio mendukung penggunaan oleh lebih dari satu pengguna. Data pada tabel *categories*, *portfolios*, *goals*, *user_accounts*, *finance_transactions*, dan *budgets* dihubungkan dengan kolom *user_id*. Setiap *query* pengolahan data juga menyertakan *user_id* dari *session* pengguna yang sedang aktif. Mekanisme tersebut digunakan agar setiap pengguna hanya dapat membaca, menambahkan, mengubah, atau menghapus data miliknya sendiri.

Password tidak disimpan dalam bentuk teks biasa. Sistem menggunakan *generate_password_hash()* saat registrasi atau penggantian *password* dan *check_password_hash()* ketika pengguna melakukan *login*. *Query* yang membawa data pengguna menggunakan placeholder *%s* beserta *tuple* parameter. Dengan cara tersebut, *input* pengguna tidak dimasukkan secara langsung ke dalam struktur perintah SQL sehingga risiko *SQL Injection* dapat dikurangi.

JavaScript pada FynaCchio digunakan sebagai lapisan interaksi antarmuka dan bukan sebagai pengganti logika bisnis. JavaScript menangani modal dan *toast* Bootstrap, *checkbox* untuk aksi massal, *sorting* tabel tiga keadaan, penyimpanan posisi *scroll* menggunakan *sessionStorage*, serta *preview* dan kompresi gambar avatar melalui elemen *canvas*. Gambar avatar diperkecil hingga maksimal 150 piksel, kemudian diubah menjadi format JPEG dengan kualitas 0,8 sebelum dikirimkan kepada *server*. JavaScript juga digunakan untuk mengatur visualisasi grafik melalui *Chart.js*.

Seluruh proses inti, seperti autentikasi, CRUD, validasi aturan bisnis, perhitungan keuangan, transfer dana, pengelolaan investasi, dan akses *database*, tetap ditangani oleh Python dan Flask pada sisi server. Tombol *logout* hanya menjalankan *session.clear()* untuk menghapus *session* pengguna. Proses tersebut tidak menghentikan *server* Flask sehingga aplikasi tetap dapat melayani *request* dari pengguna lain. Sementara itu, proses ekspor laporan arus kas dilakukan menggunakan ReportLab dan hasilnya dikirimkan kepada *browser* dalam format PDF.



Gambar 2.1 Arsitektur aplikasi web FynaCchio

Struktur Folder

```

FynaCchio/
|---- app.py           # routing, autentikasi, session, CRUD, validasi, dan logika bisnis
|---- config.py        # konfigurasi Flask, MySQL, SECRET_KEY, dan akun awal
|---- database.py      # koneksi serta inialisasi database
|---- analytics.py     # class dan perhitungan analitik finansial
|---- utils.py         # format Rupiah, kuantitas aset, dan pembersihan string
|---- templates_content.py # template HTML/Jinja2, CSS, Bootstrap, JavaScript, dan Chart.js
|---- schema.sql       # rancangan 7 tabel database MySQL
|---- requirements.txt  # Flask, connector, ReportLab, Werkzeug
|---- README.md        # petunjuk instalasi dan menjalankan aplikasi
|---- static           # aset statis
|   |---- logo1.svg    # logo FynaCchio 1
|   |---- logo2.svg    # logo FynaCchio 2
  
```

Gambar 2.2 Struktur folder proyek FynaCchio

Daftar Fungsi dan Metode

Setiap berkas dalam FynaCchio memiliki fungsi yang berbeda, tetapi tetap saling terhubung dalam menjalankan aplikasi. `app.py` menangani *route*, autentikasi, validasi, dan proses pengolahan data. Berkas tersebut menggunakan fungsi dari `database.py` untuk mengakses MySQL, `analytics.py` untuk melakukan perhitungan finansial, serta `utils.py` untuk memformat dan membersihkan data sebelum ditampilkan kepada pengguna.

Pembagian fungsi ke dalam beberapa modul membuat kode tidak terpusat pada satu berkas. Fungsi yang berkaitan dengan koneksi *database*, analisis keuangan, pemformatan data, dan pengelolaan antarmuka ditempatkan sesuai dengan tanggung jawabnya. Dengan

demikian, proses pemeriksaan, perbaikan, dan pengembangan aplikasi dapat dilakukan dengan lebih terarah.

Berdasarkan struktur kode yang digunakan, FynaCchio memiliki 40 fungsi dan metode. Tabel 2.1 sampai dengan Tabel 2.9 menjelaskan nama fungsi dan metode, parameter atau input beserta perannya, nilai balik, jenis, dan tugas utamanya. Input pada route Flask dapat berasal dari `request.form`, `request.args`, `session`, atau parameter URL. Parameter `self` pada metode kelas tidak dicantumkan karena secara otomatis merujuk pada objek yang sedang digunakan.

Seluruh *route* Flask dikategorikan sebagai non-void karena selalu memberikan nilai balik kepada browser. Nilai balik tersebut dapat berupa halaman HTML yang dihasilkan melalui `render_template_string()`, `redirect` menuju halaman lain, atau file PDF yang dibuat menggunakan ReportLab.

Tabel 2.1 Fungsi pada modul database.py

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
1	<code>configure_database(app)</code>	<code>app</code> : objek aplikasi Flask yang menyediakan konfigurasi koneksi database dan disimpan agar dapat digunakan oleh fungsi lain dalam <code>database.py</code> .	None	Void	Menyimpan objek aplikasi Flask agar konfigurasi MySQL dapat digunakan oleh fungsi dalam modul database.
2	<code>get_db_connection()</code>	Tidak memiliki parameter langsung. Fungsi menggunakan konfigurasi <code>DB_HOST</code> , <code>DB_USER</code> , <code>DB_PASS</code> , dan <code>DB_NAME</code> dari objek aplikasi Flask.	MySQLConnection atau None	Non-Void	Membuka koneksi ke MySQL berdasarkan konfigurasi aplikasi dan menangani kegagalan koneksi.
3	<code>init_db()</code>	Tidak memiliki parameter langsung. Fungsi menggunakan konfigurasi database aplikasi untuk membuat database, tabel, dan menyesuaikan struktur database lama.	None	Void	Membuat database dan tujuh tabel jika belum tersedia serta menyesuaikan beberapa kolom pada database lama.

Tabel 2.2 Fungsi pada modul *analytics.py*

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
4	<code>BaseFinance.__init__(user_id)</code>	<code>user_id</code> : ID pengguna yang disimpan sebagai atribut objek untuk mengidentifikasi pemilik data keuangan.	None	Constructor	Menyimpan ID pengguna sebagai atribut kelas induk.
5	<code>FinancialAnalyzer.__init__(user_id)</code>	<code>user_id</code> : ID pengguna yang diteruskan kepada konstruktor <code>BaseFinance</code> dan digunakan sebagai identitas pemilik data yang dianalisis.	None	Constructor	Memanggil konstruktor kelas induk untuk mewarisi atribut <code>user_id</code> .
6	<code>FinancialAnalyzer.evaluate_health(income, expense)</code>	<code>income</code> : total pemasukan pengguna <code>expense</code> : total pengeluaran pengguna yang dibandingkan untuk menentukan kondisi kesehatan finansial.	str	Non-Void	Menentukan kondisi kesehatan finansial berdasarkan perbandingan pemasukan dan pengeluaran.
7	<code>FinancialAnalyzer.calculate_savings_ratio(income, expense)</code>	<code>income</code> : total pemasukan pengguna <code>expense</code> : total pengeluaran yang dikurangkan dari pemasukan untuk menghitung rasio tabungan.	int	Non-Void	Menghitung persentase rasio tabungan pengguna dengan nilai minimum 0%.

Tabel 2.3 Fungsi pada modul *utils.py*

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
8	<code>format_rupiah(nominal)</code>	<code>nominal</code> : nilai angka yang akan diubah menjadi teks mata uang Rupiah.	str	Non-Void	Mengubah angka menjadi teks mata uang Rupiah dengan tanda titik sebagai pemisah ribuan.
9	<code>format_qty(value, asset_type)</code>	<code>value</code> : jumlah atau kuantitas aset <code>asset_type</code> : jenis aset yang menentukan	str	Non-Void	Memformat kuantitas aset dan menambahkan satuan Lot, Gram, Koin, atau Unit.

		satuan Lot, Gram, Koin, atau Unit.			
10	<code>clean_string_input(raw_input)</code>	<code>raw_input</code> : teks mentah yang akan dibersihkan dari spasi pada awal dan akhir serta diatur kapitalisasi setiap katanya.	<code>str</code>	Non-Void	Menghapus spasi pada awal dan akhir teks serta mengatur kapitalisasi awal setiap kata.

Tabel 2.4 Fungsi autentikasi dan profil pada modul `app.py`

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
11	<code>login_required(f)</code>	<code>f</code> : fungsi route asli yang akan dilindungi dengan pemeriksaan session pengguna.	<code>decorated_function</code>	Non-Void/ Decorator	Membungkus route agar hanya dapat diakses oleh pengguna dengan session yang masih valid.
12	<code>decorated_function(args, kwargs)</code>	<code>args</code> : kumpulan argumen posisi yang diteruskan kepada fungsi route asli <code>kwargs</code> : kumpulan argumen bernama yang diteruskan kepada route, termasuk parameter dari URL apabila tersedia. Fungsi juga membaca session untuk memeriksa pengguna.	Flask response	Non-Void	Memeriksa session dan keberadaan pengguna pada database sebelum menjalankan route asli.
13	<code>inject_user_profile()</code>	Tidak memiliki parameter langsung. Fungsi membaca <code>user_id</code> dari <code>session</code> untuk mengambil profil pengguna yang sedang aktif.	<code>dict</code>	Non-Void	Menyediakan username, nama tampilan, avatar, waktu server dijalankan, dan jumlah session aktif ke seluruh template.
14	<code>register()</code>	Form <code>display_name</code> : nama pengguna yang ditampilkan pada aplikasi	HTML atau redirect response	Non-Void	Mendaftarkan pengguna, membuat hash password, serta menambahkan

		username : identitas unik untuk login password : kata sandi yang akan diamankan dalam bentuk hash.			kategori, platform, dan akun bawaan.
15	login()	Form username : identitas pengguna yang dicari pada database password : kata sandi yang diverifikasi dengan hash pada database.	HTML atau redirect response	Non-Void	Memverifikasi username dan password, kemudian menyimpan identitas pengguna ke dalam session.
16	logout()	Tidak memiliki parameter langsung. Fungsi menggunakan session pengguna yang sedang aktif untuk menghapus data login.	Redirect response	Non-Void	Menghapus session pengguna dan mengarahkannya ke halaman login tanpa menghentikan server.
17	edit_profile()	Form display_name : nama tampilan baru username : identitas login baru password : kata sandi baru yang bersifat opsional avatar_base64 : data gambar avatar dalam format Base64.	Redirect response	Non-Void	Memperbarui nama tampilan, username, password, dan avatar pengguna.

Tabel 2.5 Fungsi dashboard dan pengolahan data pada modul app.py

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
18	dashboard()	Query year : tahun kalender transaksi yang ditampilkan month : bulan kalender transaksi trend_view : jenis tampilan grafik, yaitu harian, bulanan, atau tahunan trend_month : bulan yang digunakan pada grafik tren	HTML hasil render_template_string()	Non-Void	Menghitung ringkasan pemasukan, pengeluaran, saldo, aset, anggaran, kalender, grafik, dan kesehatan finansial.

		trend_year_y : tahun yang digunakan untuk grafik tahunan trend_year_m : tahun yang digunakan untuk grafik bulanan atau harian start_date : tanggal awal rentang khusus end_date : tanggal akhir rentang khusus.			
19	manage_transactions()	Query search : kata kunci pencarian deskripsi atau kategori period : periode transaksi dalam format tahun dan bulan account : nama akun yang digunakan sebagai filter category_id : ID kategori transaksi yang dipilih type : jenis transaksi pemasukan atau pengeluaran.	HTML hasil render_template_string()	Non-Void	Menampilkan transaksi keuangan berdasarkan pencarian dan filter pengguna.
20	get_account_balance(conn, account_name, user_id, exclude_tx_id=None)	conn : koneksi MySQL yang digunakan untuk menjalankan query account_name : nama akun yang saldonya dihitung user_id : ID pemilik akun dan transaksi exclude_tx_id : ID transaksi yang dikecualikan dari perhitungan, terutama saat proses edit.	float	Non-Void	Menghitung saldo berjalan suatu akun dan dapat mengecualikan transaksi tertentu saat proses edit.
21	get_asset_qty(conn, portfolio_id, asset_type, asset_name, user_id)	conn : koneksi MySQL portfolio_id : ID platform investasi tempat aset disimpan asset_type : jenis aset yang dihitung asset_name : nama aset yang dihitung	float	Non-Void	Menghitung kuantitas aset yang masih dimiliki berdasarkan riwayat pembelian dan penjualan.

		<code>user_id</code> : ID pengguna pemilik aset.			
--	--	--	--	--	--

Tabel 2.6 Fungsi transaksi dan akun pada modul `app.py`

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
22	<code>add_transaction()</code>	Form <code>tgl</code> : tanggal transaksi <code>type</code> : jenis transaksi pemasukan atau pengeluaran <code>account</code> : akun yang digunakan <code>category_id</code> : kategori transaksi <code>nominal</code> : jumlah uang dalam transaksi <code>description</code> : keterangan transaksi.	Redirect response	Non-Void	Menambahkan transaksi baru setelah memvalidasi nominal dan ketersediaan saldo akun.
23	<code>edit_transaction()</code>	Form <code>tx_id</code> : ID transaksi yang akan diubah <code>tgl</code> : tanggal transaksi baru <code>type</code> : jenis transaksi baru <code>account</code> : akun yang digunakan <code>category_id</code> : kategori transaksi <code>nominal</code> : jumlah transaksi baru <code>description</code> : keterangan transaksi baru.	Redirect response	Non-Void	Memperbarui transaksi terpilih setelah melakukan validasi nominal dan saldo.
24	<code>transfer_funds()</code>	Form <code>tgl</code> : tanggal transfer <code>from_account</code> : akun sumber dana <code>to_account</code> : akun tujuan dana <code>nominal</code> : jumlah dana yang ditransfer <code>description</code> : catatan atau keterangan transfer.	Redirect response	Non-Void	Mencatat transfer antar akun sebagai pasangan transaksi pengeluaran dan pemasukan.
25	<code>bulk_action()</code>	Form <code>tx_ids</code> : daftar ID transaksi yang dipilih, kemudian divalidasi sebagai angka dan dimasukkan ke dalam <code>set</code> agar setiap ID bersifat unik <code>action</code> : tindakan yang akan dijalankan terhadap transaksi, yaitu penghapusan.	Redirect response	Non-Void	Menghapus satu atau beberapa transaksi yang dipilih oleh pengguna.

26	<code>add_account()</code>	Form name : nama akun bank, uang tunai, atau dompet digital yang akan ditambahkan.	Redirect response	Non-Void	Menambahkan akun bank, uang tunai, atau dompet digital milik pengguna.
27	<code>delete_account(id)</code>	<code>id</code> : ID akun dari URL yang digunakan untuk menentukan akun yang akan dihapus.	Redirect response	Non-Void	Menghapus akun pengguna berdasarkan ID tanpa menghapus riwayat transaksi sebelumnya.

Tabel 2.7 Fungsi portfolio dan investasi pada modul `app.py`

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
28	<code>manage_portfolios()</code>	Query <code>search</code> : kata kunci pencarian aset atau target tabungan <code>period</code> : periode transaksi investasi <code>account</code> : akun sumber atau penerima dana <code>platform</code> : ID platform investasi yang dipilih <code>type</code> : jenis transaksi investasi yang digunakan sebagai filter.	HTML hasil <code>render_template_string()</code>	Non-Void	Menampilkan platform investasi, target tabungan, aset aktif, dan riwayat investasi berdasarkan filter.
29	<code>add_portfolio_tx()</code>	Form <code>tgl</code> : tanggal transaksi investasi <code>type</code> : jenis transaksi pembelian, penjualan, penyetoran, atau penarikan <code>account</code> : akun sumber atau penerima dana <code>portfolio_id</code> : ID platform investasi yang dipilih <code>goal_id</code> : ID target tabungan yang dipilih <code>asset_type</code> : jenis aset <code>asset_name</code> : nama aset <code>quantity</code> : jumlah aset yang dibeli atau dijual	Redirect response	Non-Void	Mencatat pembelian atau penjualan aset serta penyetoran atau penarikan target tabungan.

		nominal : nilai uang dalam transaksi.			
30	edit_portfolio_tx()	Form tx_id : ID transaksi investasi yang akan diubah tgl : tanggal transaksi baru type : jenis transaksi baru account : akun sumber atau penerima dana portfolio_id : ID platform investasi goal_id : ID target tabungan asset_type : jenis aset asset_name : nama aset quantity : jumlah aset nominal : nilai transaksi baru.	Redirect response	Non-Void	Memperbarui transaksi investasi atau target tabungan dan menyesuaikan kembali saldo target.
31	bulk_action_portfolio()	Form tx_ids : daftar ID transaksi investasi atau tabungan yang dipilih action : tindakan yang akan dijalankan, yaitu penghapusan transaksi.	Redirect response	Non-Void	Menghapus riwayat investasi atau tabungan terpilih dan menyesuaikan saldo target terkait.
32	add_portfolio()	Form name : nama platform investasi baru yang akan ditambahkan.	Redirect response	Non-Void	Menambahkan platform investasi baru milik pengguna.
33	delete_portfolio(id)	id : ID platform investasi dari URL yang digunakan untuk menentukan platform yang akan dihapus.	Redirect response	Non-Void	Menghapus platform berdasarkan ID dengan mempertahankan baris transaksi melalui relasi ON DELETE SET NULL.

Tabel 2.8 Fungsi anggaran dan target tabungan pada modul app.py

No.	Nama Fungsi/Metode	Parameter/Input dan Perannya	Nilai Balik	Jenis	Tugas Utama
-----	--------------------	------------------------------	-------------	-------	-------------

34	<code>manage_budgets()</code>	Query period : periode anggaran yang akan ditampilkan dalam format tahun dan bulan.	HTML hasil <code>render_template_string()</code>	Non-Void	Menampilkan anggaran, realisasi pengeluaran, sisa anggaran, dan batas pengeluaran harian.
35	<code>add_goal()</code>	Form name : nama target tabungan target_amount : jumlah uang yang ingin dicapai deadline : tanggal batas waktu pencapaian target.	Redirect response	Non-Void	Menambahkan target tabungan setelah memvalidasi nama dan nominal target.
36	<code>delete_goal(id)</code>	id : ID target tabungan dari URL yang digunakan untuk menentukan target yang akan dihapus.	Redirect response	Non-Void	Menghapus target tabungan milik pengguna berdasarkan ID.
37	<code>add_budget()</code>	Form category_id : ID kategori pengeluaran yang diberi anggaran period : periode berlakunya anggaran dalam format tahun dan bulan; amount: jumlah batas anggaran.	Redirect response	Non-Void	Menambahkan anggaran baru atau memperbarui anggaran pada kategori dan periode yang sama.
38	<code>delete_budget(id)</code>	id: ID anggaran dari URL yang digunakan untuk menentukan anggaran yang akan dihapus.	Redirect response	Non-Void	Menghapus anggaran kategori milik pengguna berdasarkan ID.

Tabel 2.9 Fungsi laporan pada modul `app.py`

No.	Nama Fungsi/Metode	Parameter	Nilai Balik	Jenis	Tugas Utama
39	<code>reports()</code>	Query month : periode bulan dan tahun yang digunakan untuk menyusun serta menampilkan	HTML hasil <code>render_template_string()</code>	Non-Void	Menyusun buku besar bulanan yang berisi debit, kredit, dan saldo berjalan.

		laporan buku besar.			
40	<code>export_pdf()</code>	Query month : periode bulan dan tahun laporan yang akan dibuat dan diunduh dalam format PDF.	File response PDF	Non-Void	Membuat dan mengunduh laporan arus kas bulanan dalam format PDF menggunakan ReportLab.

Penggunaan Struktur Data Koleksi

FynaCchio menggunakan empat jenis struktur data koleksi bawaan Python, yaitu `list`, `dictionary`, `tuple`, dan `set`. Struktur `list` digunakan untuk menyimpan data berurutan, seperti hasil *query*, daftar akun, parameter filter, dan riwayat transaksi. Struktur `dictionary` digunakan untuk menyimpan data dalam bentuk pasangan kunci dan nilai, seperti filter halaman, saldo platform, data kepemilikan aset, serta data grafik. Struktur `tuple` digunakan untuk mengirimkan parameter pada *query* SQL agar data dapat diproses secara terstruktur dan aman.

Struktur `set` digunakan dalam fungsi `bulk_action()` untuk menampung ID numerik transaksi yang dipilih ketika penghapusan massal dilakukan. Setiap ID yang valid dimasukkan menggunakan metode `add()`. Karena `set` hanya menyimpan nilai unik, ID transaksi yang sama tidak dapat diproses lebih dari satu kali. Kumpulan ID tersebut kemudian diurutkan dan dikonversi menjadi `tuple` sebelum digunakan sebagai parameter dalam *query* penghapusan.

Pembagian tersebut menunjukkan bahwa setiap modul dan kelompok fungsi memiliki tanggung jawab yang berbeda. Fungsi koneksi *database*, analisis keuangan, pemformatan data, autentikasi, transaksi, investasi, anggaran, dan laporan dipisahkan sesuai kegunaannya. Struktur tersebut membuat kode lebih mudah dibaca, diperiksa, dan dikembangkan tanpa harus mengubah seluruh bagian aplikasi.

2.2 Skema Database SQL

FynaCchio menggunakan basis data MySQL dengan nama `fynacchio`. Nama basis data tersebut ditentukan melalui konfigurasi `DB_NAME` pada `config.py`. Proses pembuatan basis

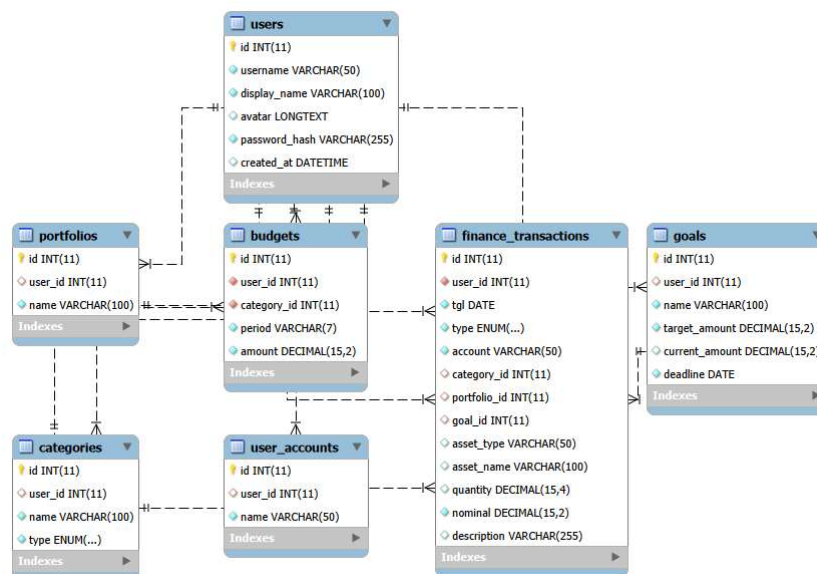
data dan tabel dijalankan oleh fungsi `init_db()` dalam `database.py`, sedangkan rancangan SQL yang sama disediakan secara terpisah dalam `schema.sql`. Perintah `CREATE DATABASE IF NOT EXISTS` dan `CREATE TABLE IF NOT EXISTS` digunakan agar proses inisialisasi tidak membuat ulang *database* atau tabel yang telah tersedia.

Database FynaCchio terdiri atas 7 tabel, yaitu `users`, `categories`, `portfolios`, `goals`, `user_accounts`, `finance_transactions`, dan `budgets`. Seluruh tabel memiliki kolom `id` bertipe `INT` sebagai *Primary Key* (PK) dengan `AUTO_INCREMENT`. Hubungan antartabel dibentuk menggunakan 10 Foreign Key (FK).

Tabel `users` menjadi tabel induk utama karena data pada 6 tabel lainnya dipisahkan berdasarkan pengguna melalui kolom `user_id`. Sementara itu, `finance_transactions` menjadi tabel pusat pencatatan keuangan karena digunakan untuk menyimpan transaksi reguler, transfer antar akun, pembelian atau penjualan aset, serta penyetoran atau penarikan target tabungan.

Entity Relationship Diagram

Hubungan antara 7 tabel dalam *database* FynaCchio ditampilkan melalui ERD pada Gambar 2.3. Garis relasi pada ERD menunjukkan hubungan satu-ke-banyak atau *one-to-many* (1:N). Artinya, satu data pada tabel induk dapat berhubungan dengan lebih dari satu data pada tabel anak.



Gambar 2.3 ERD database fynacchio

Pada ERD yang dihasilkan melalui MySQL Workbench, tipe data INT dapat ditampilkan sebagai INT(11). Penulisan tersebut tetap merujuk pada tipe data INT yang digunakan dalam `schema.sql` dan `database.py`. Angka 11 merupakan lebar tampilan yang ditampilkan oleh MySQL Workbench dan bukan jumlah digit maksimum yang dapat disimpan.

Struktur Tabel Database

Rancangan tujuh tabel FynaCchio memiliki total 40 kolom. Struktur lengkap setiap tabel, tipe data, batasan, dan kegunaan kolom disajikan pada Tabel 2.10.

Tabel 2.10 Rancangan struktur tabel database FynaCchio

No.	Tabel	Kolom	Tipe Data	Kunci dan Batasan	Kegunaan
1	users	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik setiap pengguna.
2	users	username	VARCHAR(50)	NOT NULL, UNIQUE	Menyimpan identitas unik yang digunakan untuk login.
3	users	display_name	VARCHAR(100)	NOT NULL	Menyimpan nama pengguna yang ditampilkan pada aplikasi.
4	users	avatar	LONGTEXT	NULL diperbolehkan	Menyimpan gambar avatar pengguna dalam bentuk teks Base64.
5	users	password_hash	VARCHAR(255)	NOT NULL	Menyimpan hasil hash kata sandi pengguna.
6	users	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Menyimpan waktu pembuatan akun secara otomatis.
7	categories	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik kategori transaksi.
8	categories	user_id	INT	FK ke users.id, NULL diperbolehkan, ON DELETE CASCADE	Menentukan pengguna pemilik kategori.

9	categories	name	VARCHAR(100)	NOT NULL	Menyimpan nama kategori transaksi.
10	categories	type	ENUM('Pemasukan', 'Pengeluaran')	NOT NULL	Menentukan apakah kategori digunakan untuk pemasukan atau pengeluaran.
11	portfolios	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik platform investasi.
12	portfolios	user_id	INT	FK ke users.id, NULL diperbolehkan, ON DELETE CASCADE	Menentukan pengguna pemilik platform investasi.
13	portfolios	name	VARCHAR(100)	NOT NULL	Menyimpan nama platform investasi.
14	goals	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik target tabungan.
15	goals	user_id	INT	FK ke users.id, NULL diperbolehkan, ON DELETE CASCADE	Menentukan pengguna pemilik target tabungan.
16	goals	name	VARCHAR(100)	NOT NULL	Menyimpan nama target tabungan.
17	goals	target_amount	DECIMAL(15,2)	NOT NULL	Menyimpan nominal yang ingin dicapai.
18	goals	current_amount	DECIMAL(15,2)	DEFAULT 0	Menyimpan nominal tabungan yang telah terkumpul.
19	goals	deadline	DATE	NOT NULL	Menyimpan tanggal batas pencapaian target.
20	user_accounts	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik akun keuangan.
21	user_accounts	user_id	INT	FK ke users.id, NULL diperbolehkan, ON DELETE CASCADE	Menentukan pengguna pemilik akun.
22	user_accounts	name	VARCHAR(50)	NOT NULL	Menyimpan nama bank, uang tunai, atau dompet digital.

23	finance_transactions	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik setiap transaksi keuangan.
24	finance_transactions	user_id	INT	FK ke users.id, NOT NULL, ON DELETE CASCADE	Menentukan pengguna pemilik transaksi.
25	finance_transactions	tgl	DATE	NOT NULL	Menyimpan tanggal transaksi.
26	finance_transactions	type	ENUM('Pemasukan', 'Pengeluaran')	NOT NULL	Menentukan arah arus kas transaksi.
27	finance_transactions	account	VARCHAR(50)	NOT NULL	Menyimpan nama akun yang digunakan dalam transaksi.
28	finance_transactions	category_id	INT	FK ke categories.id, NULL, ON DELETE SET NULL	Menghubungkan transaksi reguler dengan kategori
29	finance_transactions	portfolio_id	INT	FK ke portfolios.id, NULL, ON DELETE SET NULL	Menghubungkan transaksi investasi dengan platform.
30	finance_transactions	goal_id	INT	FK ke goals.id, NULL, ON DELETE SET NULL	Menghubungkan transaksi tabungan dengan target keuangan.
31	finance_transactions	asset_type	VARCHAR(50)	NULL diperbolehkan	Menyimpan jenis aset, seperti Saham, Reksa Dana, Emas, Kripto, atau Lainnya.
32	finance_transactions	asset_name	VARCHAR(100)	NULL diperbolehkan	Menyimpan nama aset atau judul simpanan.
33	finance_transactions	quantity	DECIMAL(15,4)	NULL diperbolehkan	Menyimpan kuantitas aset dengan maksimal empat angka desimal.
34	finance_transactions	nominal	DECIMAL(15,2)	NOT NULL	Menyimpan nilai uang pada transaksi.
35	finance_transactions	description	VARCHAR(255)	NULL diperbolehkan	Menyimpan keterangan transaksi.

36	budgets	id	INT	PK, AUTO_INCREMENT	Menyimpan ID unik anggaran.
37	budgets	user_id	INT	FK ke users.id, NOT NULL, ON DELETE CASCADE	Menentukan pengguna pemilik anggaran.
38	budgets	category_id	INT	FK ke categories.id, NOT NULL, ON DELETE CASCADE	Menentukan kategori pengeluaran yang diberi anggaran.
39	budgets	period	VARCHAR(7)	NOT NULL	Menyimpan periode anggaran dalam format YYYY-MM
40	budgets	amount	DECIMAL(15,2)	NOT NULL	Menyimpan jumlah batas anggaran kategori.

Tipe DECIMAL(15,2) digunakan pada data nominal agar angka dapat menyimpan maksimal 15 digit dengan 2 angka di belakang koma. Tipe DECIMAL(15,4) digunakan pada quantity karena kuantitas aset dapat memiliki nilai pecahan hingga empat angka di belakang koma. Tipe ENUM membatasi nilai agar hanya dapat berisi Pemasukan atau Pengeluaran. Sementara itu, VARCHAR(7) pada kolom period digunakan untuk menyimpan periode dalam format tahun dan bulan, misalnya 2026-08.

Primary Key, Foreign Key, dan Relasi Tabel

Setiap tabel memiliki satu PK bernama id. Nilai tersebut dibuat secara otomatis menggunakan AUTO_INCREMENT, sehingga setiap baris memperoleh identitas yang berbeda. Selain 7 PK tersebut, basis data menggunakan 10 FK untuk membentuk hubungan antartabel. Rincian relasi kunci disajikan pada Tabel 2.11.

Tabel 2.11 Relasi foreign key database FynaCchio

No.	Foreign Key	Mengacu pada	Jenis Hubungan	Sifat FK	Aturan Penghapusan
1	categories.user_id	users.id	1:N	Opsional atau dapat NULL	ON DELETE CASCADE
2	portfolios.user_id	users.id	1:N	Opsional atau dapat NULL	ON DELETE CASCADE

3	goals.user_id	users.id	1:N	Opsional atau dapat NULL	ON DELETE CASCADE
4	user_accounts.user_id	users.id	1:N	Opsional atau dapat NULL	ON DELETE CASCADE
5	finance_transactions.user_id	users.id	1:N	Wajib atau NOT NULL	ON DELETE CASCADE
6	budgets.user_id	users.id	1:N	Wajib atau NOT NULL	ON DELETE CASCADE
7	finance_transactions.category_id	categories.id	1:N	Opsional atau dapat NULL	ON DELETE SET NULL
8	budgets.category_id	categories.id	1:N	Wajib atau NOT NULL	ON DELETE CASCADE
9	finance_transactions.portfolio_id	portfolios.id	1:N	Opsional atau dapat NULL	ON DELETE SET NULL
10	finance_transactions.goal_id	goals.id	1:N	Opsional atau dapat NULL	ON DELETE SET NULL

Seluruh hubungan pada *database* FynaCchio merupakan hubungan satu ke banyak (*one-to-many* atau 1:N). Tabel *users* menjadi tabel induk bagi *categories*, *portfolios*, *goals*, *user_accounts*, *finance_transactions*, dan *budgets*. Artinya, satu pengguna dapat memiliki banyak kategori, platform investasi, target tabungan, akun keuangan, transaksi, dan anggaran. Setiap data pada tabel anak dihubungkan dengan pengguna melalui kolom *user_id*.

Kolom *user_id* pada *categories*, *portfolios*, *goals*, dan *user_accounts* secara struktur SQL masih dapat bernilai NULL. Namun, ketika data dibuat melalui aplikasi, kolom tersebut selalu diisi menggunakan `session['user_id']` agar setiap data tercatat sebagai milik pengguna yang sedang aktif. Sementara itu, kolom *user_id* pada *finance_transactions* dan *budgets* ditetapkan sebagai NOT NULL, sehingga kedua tabel tersebut wajib memiliki hubungan dengan data pengguna.

Tabel *categories* juga memiliki hubungan *one-to-many* dengan *finance_transactions* dan *budgets*. Satu kategori dapat digunakan oleh banyak transaksi serta dapat mempunyai anggaran pada periode yang berbeda. Tabel *portfolios* dan *goals* masing-masing memiliki hubungan *one-to-many* dengan *finance_transactions*. Artinya, satu platform investasi dapat digunakan pada banyak transaksi investasi, sedangkan satu target tabungan dapat memiliki banyak transaksi penyetoran atau penarikan.

Aturan ON DELETE CASCADE digunakan pada hubungan `users` dengan `categories`, `portfolios`, `goals`, `user_accounts`, `finance_transactions`, dan `budgets`. Aturan tersebut menyebabkan seluruh data milik pengguna ikut dihapus apabila data pengguna dihapus. Hubungan `categories` dengan `budgets` juga menggunakan ON DELETE CASCADE, sehingga data anggaran ikut dihapus ketika kategori yang menjadi acuannya dihapus.

Berbeda dengan anggaran, hubungan `finance_transactions` dengan `categories`, `portfolios`, dan `goals` menggunakan ON DELETE SET NULL. Jika kategori, platform investasi, atau target tabungan dihapus, data transaksi tetap tersimpan, tetapi nilai `category_id`, `portfolio_id`, atau `goal_id` yang berkaitan akan diubah menjadi NULL. Aturan tersebut digunakan agar riwayat transaksi tidak ikut hilang ketika data acuannya sudah tidak tersedia.

Kolom `account` pada `finance_transactions` tidak menjadi *foreign key* yang mengacu pada `user_accounts.name`. Nama akun disimpan secara langsung pada kolom `account` dengan tipe `VARCHAR(50)`. Oleh karena itu, penghapusan akun dari tabel `user_accounts` hanya menghapusnya dari daftar akun pengguna dan tidak menghapus atau mengubah transaksi lama yang telah menggunakan nama akun tersebut.

Unique Key dan Integritas Data

Selain Primary Key dan Foreign Key, basis data FynaCchio menggunakan dua batasan unik untuk mencegah penyimpanan data yang sama. Batasan pertama terdapat pada kolom `users.username` melalui aturan UNIQUE. Aturan tersebut memastikan bahwa setiap pengguna memiliki username yang berbeda dan satu username tidak dapat digunakan oleh lebih dari satu akun.

Batasan kedua terdapat pada tabel `budgets` berupa composite unique key bernama `unique_budget_cat`. Batasan tersebut merupakan gabungan kolom `user_id`, `category_id`, dan `period`.

```
UNIQUE KEY unique_budget_cat (user_id, category_id, period)
```

Kombinasi tersebut memastikan bahwa satu pengguna hanya dapat mempunyai satu anggaran untuk kategori dan periode yang sama. Apabila pengguna kembali memasukkan

kategori dan periode yang sudah tersedia, fungsi `add_budget()` menjalankan perintah `ON DUPLICATE KEY UPDATE` untuk memperbarui nilai pada kolom `amount`. Dengan demikian, sistem tidak membuat baris anggaran baru yang memiliki pengguna, kategori, dan periode yang sama.

Penggunaan Tabel Finance Transactions

Tabel `finance_transactions` digunakan sebagai pusat penyimpanan seluruh aktivitas keuangan. Tabel tersebut dapat menyimpan transaksi pemasukan dan pengeluaran reguler, transfer antarakun, transaksi investasi, serta transaksi target tabungan. Jenis pencatatan dibedakan berdasarkan kolom relasi dan data tambahan yang digunakan.

Transaksi pemasukan dan pengeluaran reguler menggunakan `category_id` untuk menghubungkan transaksi dengan kategori. Pada jenis transaksi ini, `portfolio_id` dan `goal_id` tidak digunakan sehingga dapat bernilai `NULL`. Kolom `account` menyimpan nama akun yang digunakan, sedangkan `nominal` menyimpan nilai transaksi dan `description` menyimpan keterangannya.

Transaksi pembelian atau penjualan aset menggunakan `portfolio_id` untuk menghubungkan transaksi dengan platform investasi. Informasi aset disimpan melalui kolom `asset_type`, `asset_name`, dan `quantity`. Nilai Pengeluaran pada kolom `type` digunakan untuk mencatat pembelian aset, sedangkan nilai Pemasukan digunakan untuk mencatat penjualan aset.

Transaksi target tabungan menggunakan `goal_id` untuk menghubungkan transaksi dengan target yang dipilih. Penyetoran ke target tabungan dicatat sebagai Pengeluaran dari akun pengguna, sedangkan penarikan dari target tabungan dicatat sebagai Pemasukan ke akun pengguna. Setiap transaksi tersebut juga menyesuaikan nilai `current_amount` pada tabel `goals`.

Transfer antarakun dicatat sebagai 2 baris pada `finance_transactions`. Baris pertama berupa transaksi Pengeluaran pada akun sumber, sedangkan baris kedua berupa transaksi Pemasukan pada akun tujuan. Pencatatan tersebut menyebabkan saldo kedua akun dapat dihitung berdasarkan riwayat transaksi masing-masing.

Penggunaan satu tabel transaksi untuk beberapa aktivitas memungkinkan aplikasi menghitung pemasukan, pengeluaran, saldo akun, nilai investasi, perkembangan target tabungan, dan laporan keuangan tanpa membuat tabel transaksi yang terpisah untuk setiap fitur.

Penyesuaian Struktur Database Lama

Fungsi `init_db()` tidak hanya membuat basis data dan tabel baru, tetapi juga memeriksa struktur *database* lama agar tetap dapat digunakan oleh versi aplikasi terbaru. Pemeriksaan tersebut dilakukan sebelum penambahan kolom sehingga sistem tidak membuat kolom yang sama lebih dari satu kali.

Pada tabel `users`, fungsi `init_db()` memeriksa keberadaan kolom `display_name` dan avatar. Jika `display_name` belum tersedia, sistem menambahkan kolom tersebut dengan tipe `VARCHAR(100)` dan nilai awal Pengguna. Jika avatar belum tersedia, sistem menambahkan kolom bertipe `LONGTEXT` yang dapat bernilai `NULL`.

Fungsi `init_db()` juga memeriksa keberadaan kolom `user_id` pada struktur *database* lama. Jika kolom tersebut belum tersedia, sistem melakukan penyesuaian pada tabel yang memerlukan identitas pemilik, yaitu `categories`, `portfolios`, `goals`, `finance_transactions`, dan `budgets`. Penyesuaian tersebut dilakukan agar data dapat dipisahkan berdasarkan pengguna yang sedang menggunakan aplikasi.

Proses penyesuaian hanya menambahkan kolom yang diperlukan dan tidak menghapus data yang telah tersimpan. Pada pembuatan *database* baru, seluruh tabel, kolom, batasan unik, serta hubungan *foreign key* langsung dibentuk berdasarkan rancangan yang terdapat dalam `schema.sql` dan fungsi `init_db()`.

Berdasarkan rancangan tersebut, *database* FynaCchio telah memisahkan data pengguna, kategori, platform investasi, target tabungan, akun keuangan, transaksi, dan anggaran ke dalam tabel sesuai kegunaannya. Penggunaan *primary key*, *foreign key*, batasan unik, serta aturan `ON DELETE CASCADE` dan `ON DELETE SET NULL` menjaga hubungan dan konsistensi data. Rancangan tersebut juga mempertahankan riwayat transaksi yang masih diperlukan meskipun kategori, platform investasi, target tabungan, atau akun sudah dihapus dari daftar pengguna.

BAB III

IMPLEMENTASI & UJI COBA VISUAL

3.1 Tangkapan Layar (Screenshot) UI

The registration form contains the following fields and elements:

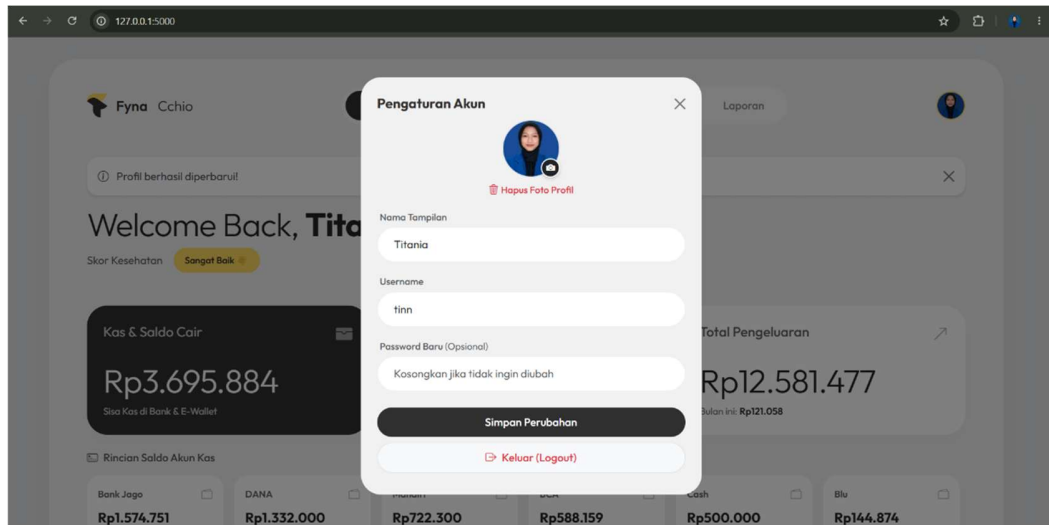
- Display Name:** Input field with placeholder text 'contoh: Budi Santoso'.
- Username:** Input field with placeholder text 'Username unik (tanpa spasi)'.
- Password:** Input field with placeholder text 'Minimal 4 karakter'.
- Buttons:** A dark blue button labeled 'Buat Akun Gratis'.
- Footer:** Text 'Sudah punya akun? [Masuk di sini](#)'.

Gambar 3.1 Halaman registrasi pengguna baru

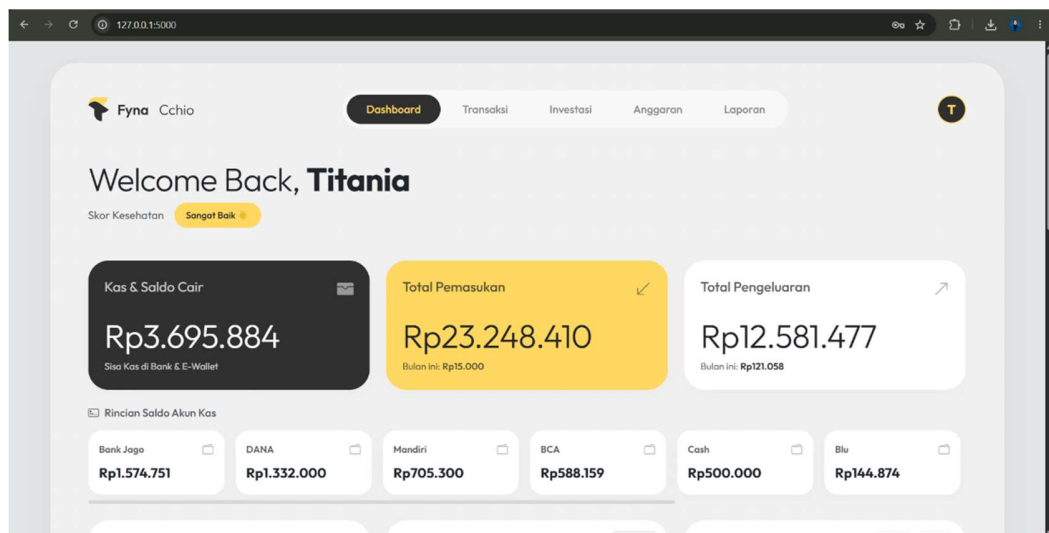
The login form contains the following fields and elements:

- Username:** Input field with placeholder text 'contoh: budi123'.
- Password:** Input field with placeholder text '.....'.
- Buttons:** A dark blue button labeled 'Login Sekarang'.
- Footer:** Text 'Belum punya akun? [Daftar di sini](#)'.

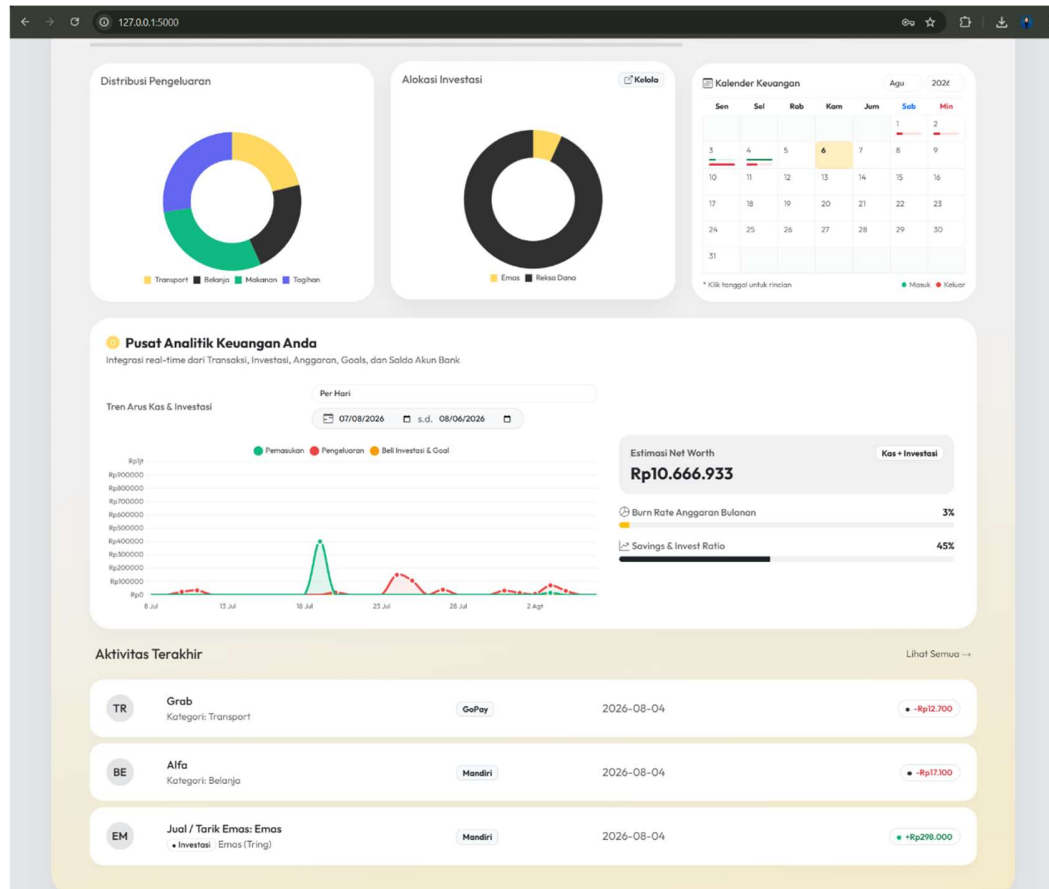
Gambar 3.2 Menu login FynaCchio



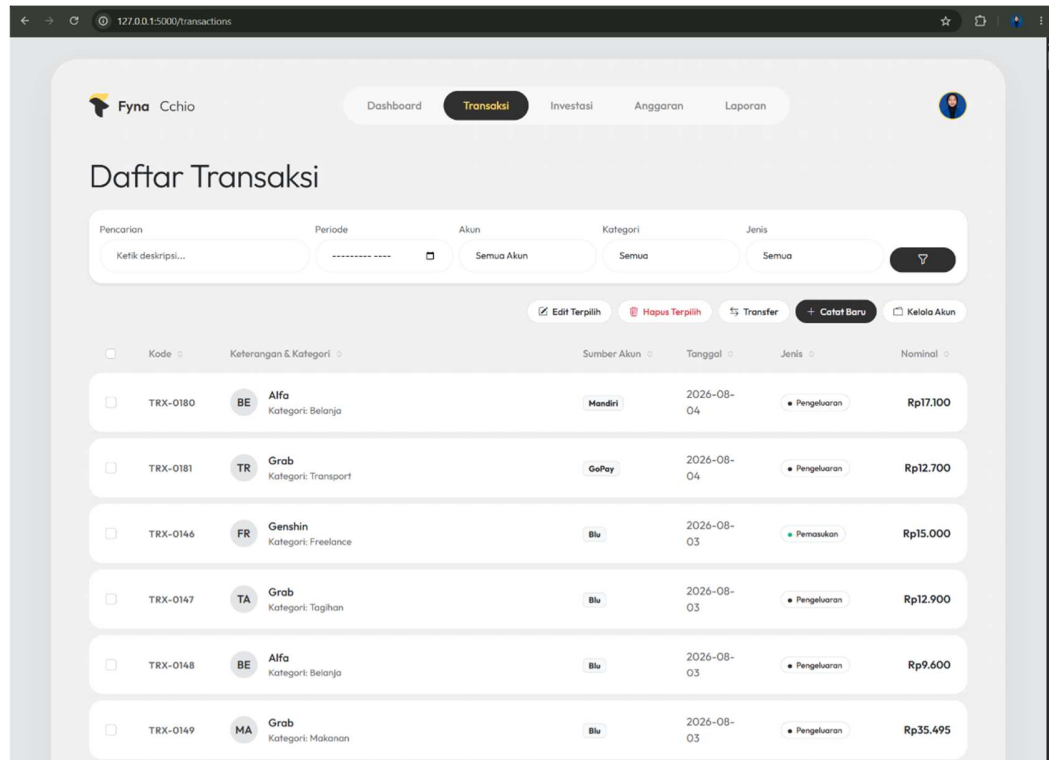
Gambar 3.3 Halaman pengeditan profil dan avatar



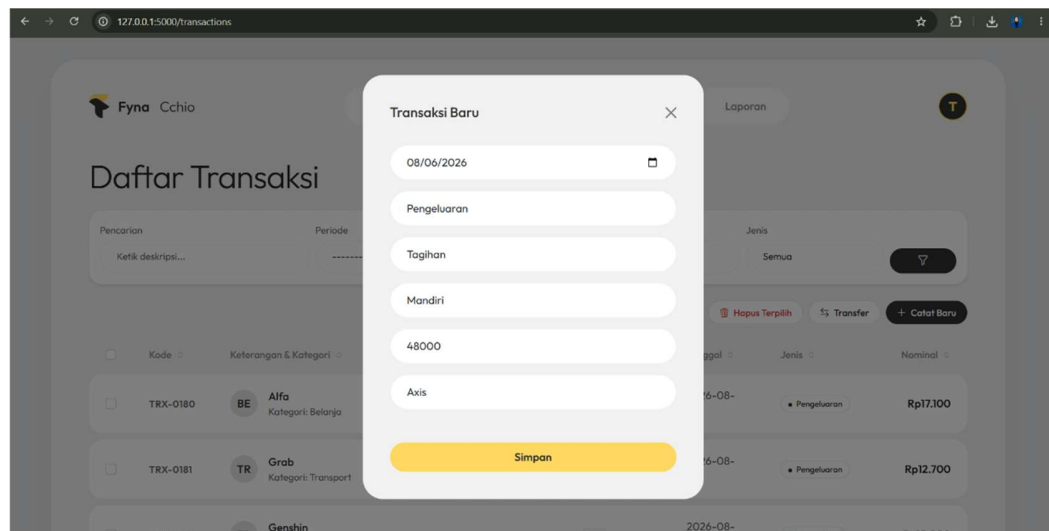
Gambar 3.4 Dashboard berisi ringkasan keuangan dan saldo akun



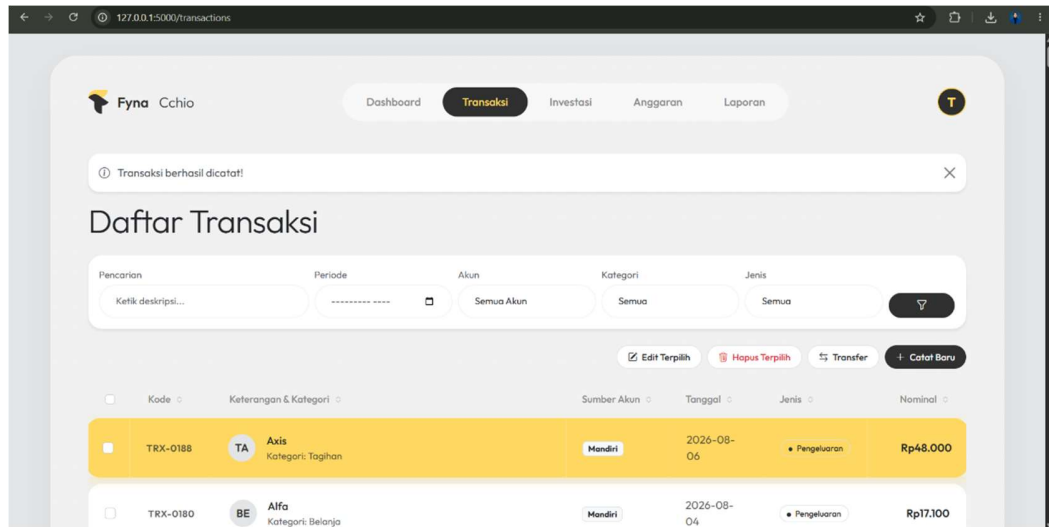
Gambar 3.5 Grafik, kalender, dan riwayat transaksi pada dashboard



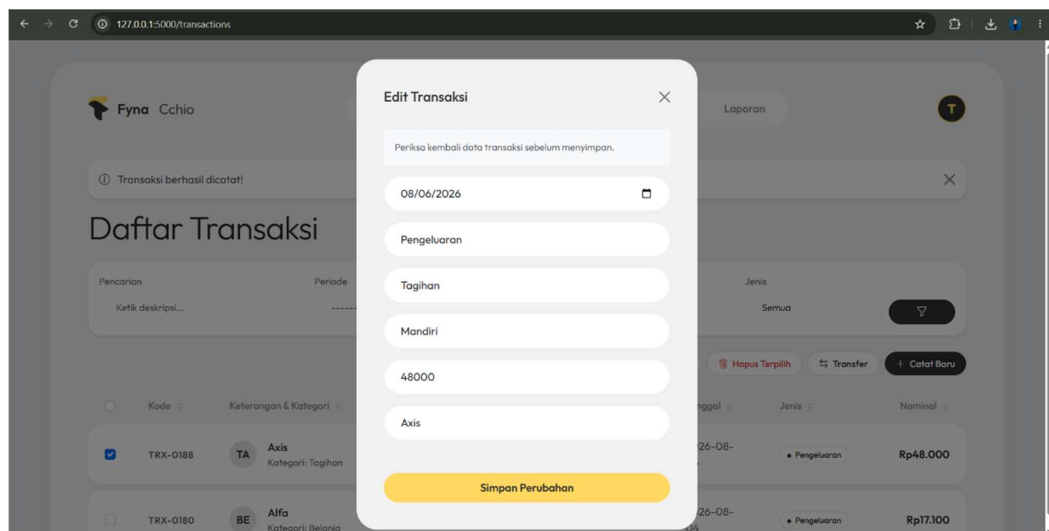
Gambar 3.6 Halaman daftar transaksi



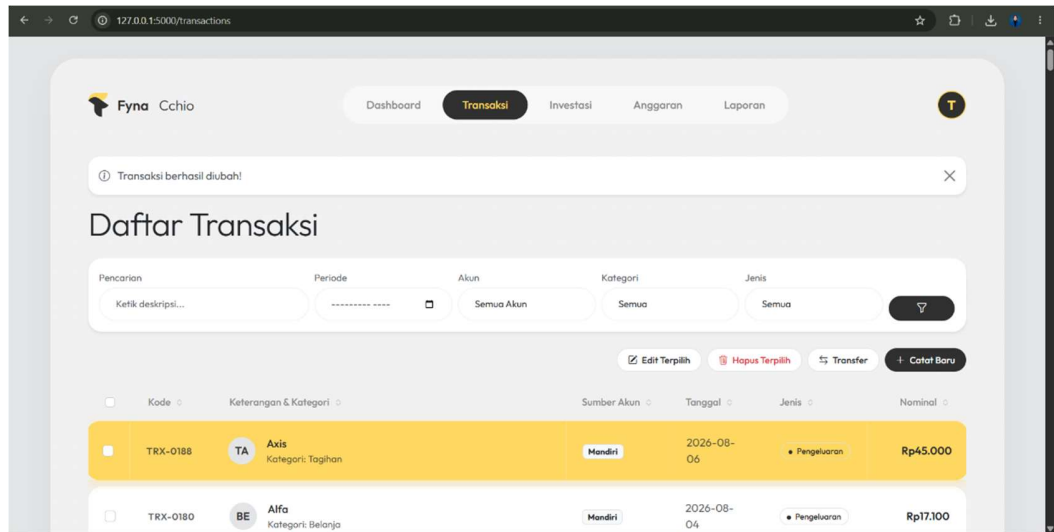
Gambar 3.7 Form penambahan transaksi



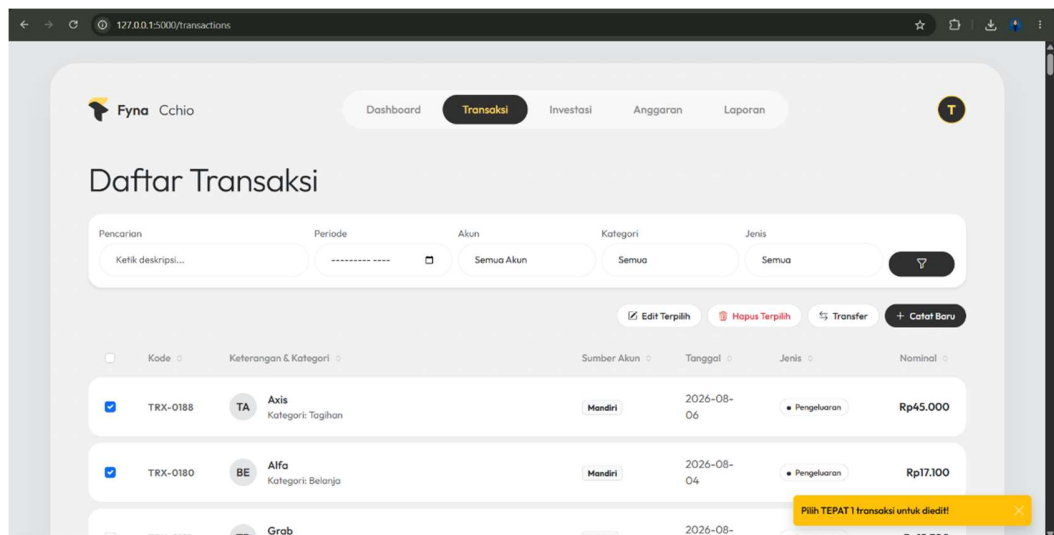
Gambar 3.8 Hasil penambahan transaksi



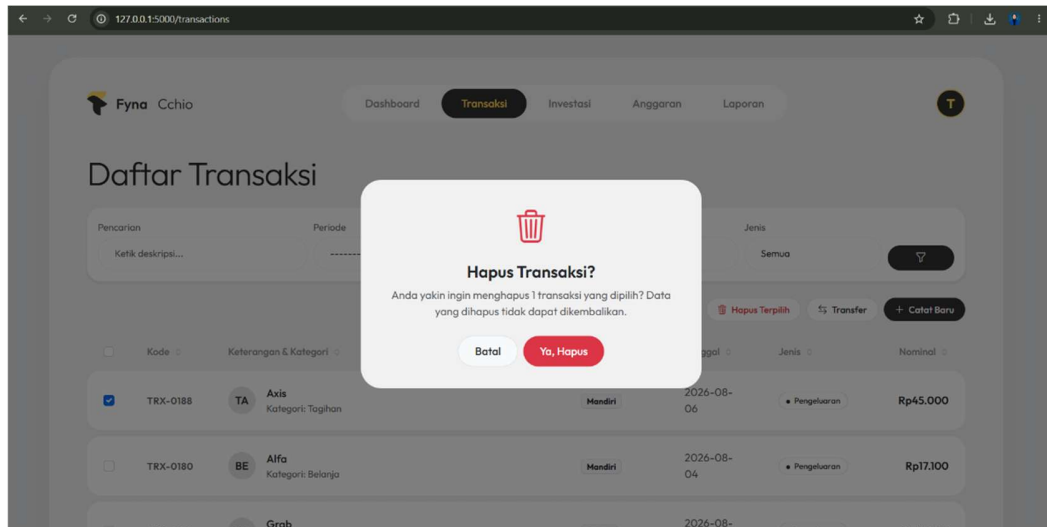
Gambar 3.9 Form pengeditan transaksi



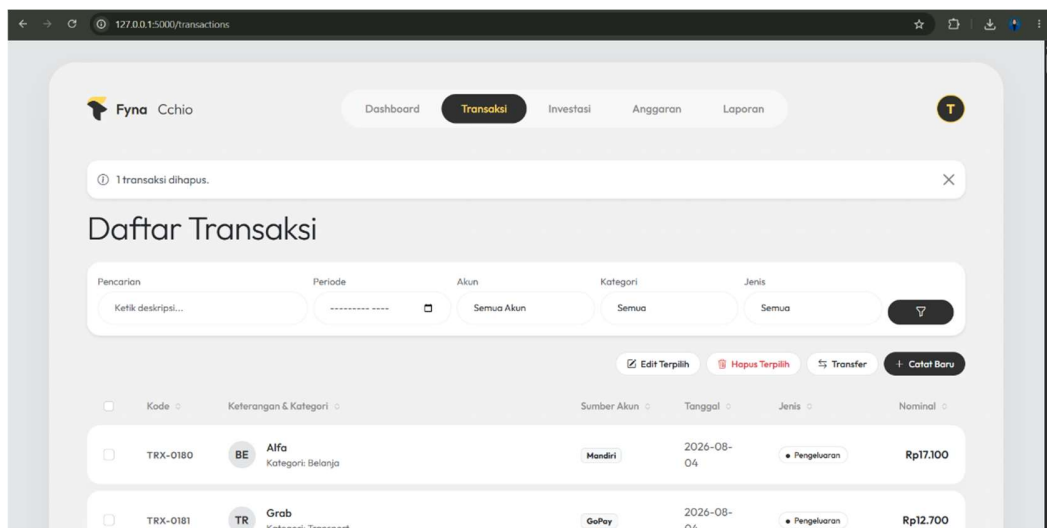
Gambar 3.10 Hasil pengeditan transaksi



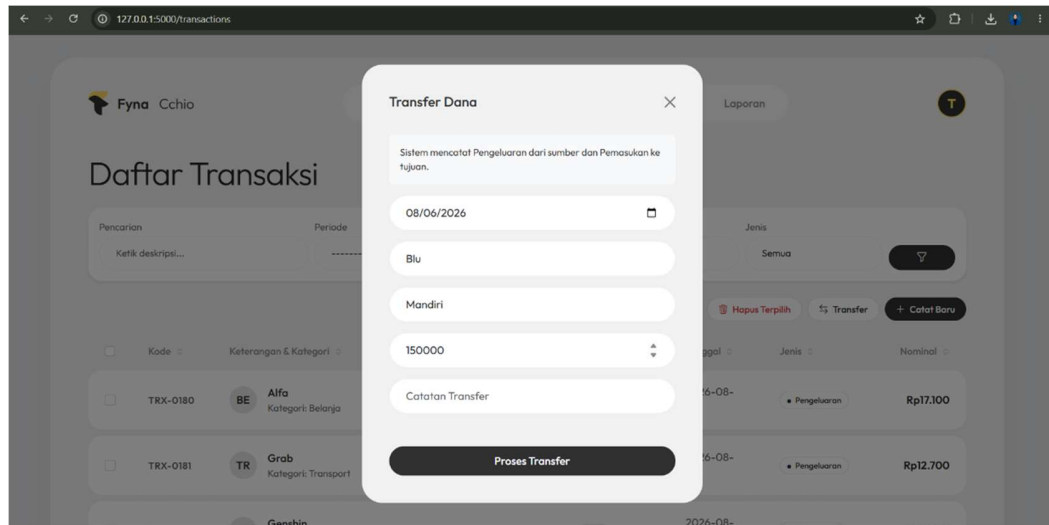
Gambar 3.11 Pemilihan transaksi yang akan dihapus



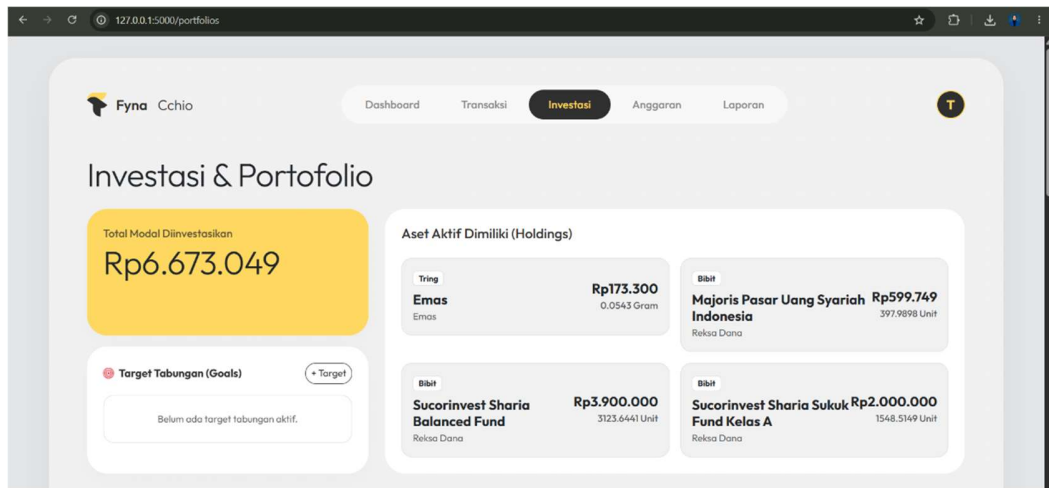
Gambar 3.12 Konfirmasi penghapusan transaksi



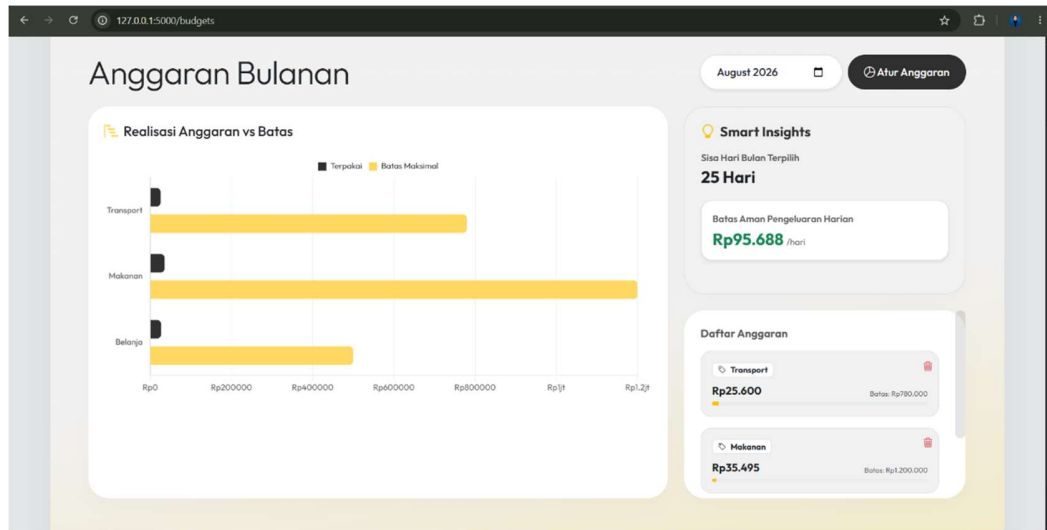
Gambar 3.13 Hasil penghapusan transaksi



Gambar 3.14 Form transfer dana antar akun



Gambar 3.15 Ringkasan portofolio, aset, dan target tabungan



Gambar 3.18 Halaman anggaran dan realisasi pengeluaran

Buku Besar Akuntansi

August 2026 [PDF](#)

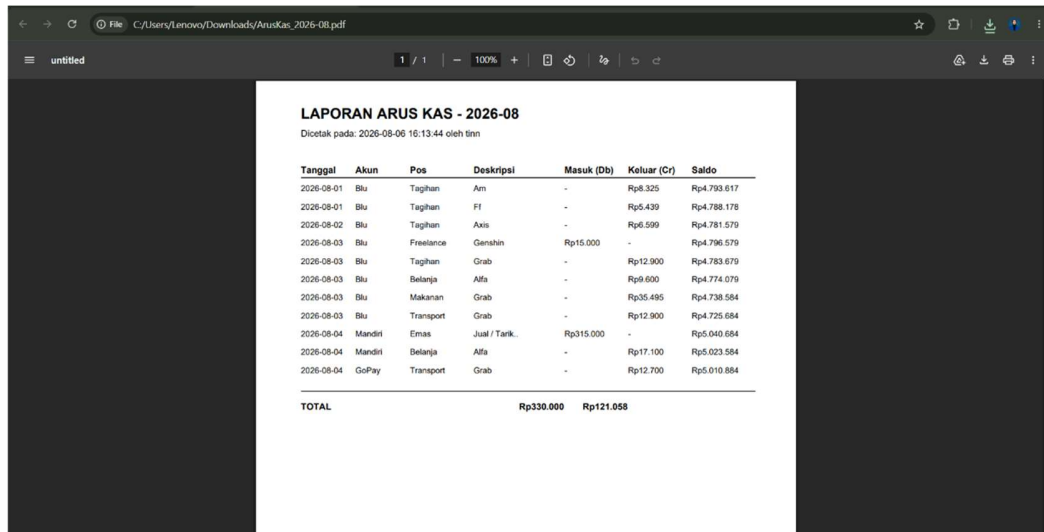
Kas Masuk (Debit): **+Rp330.000**

Kas Keluar (Kredit): **-Rp121.058**

Net Cashflow: **Rp208.942**

Tanggal	Akun Bank	Keterangan & Pos	Pemasukan (Debit)	Pengeluaran (Kredit)	Saldo Berjalan
2026-08-04	GoPay	Grab Transport	-	Rp12.700	Rp5.010.884
2026-08-04	Mandiri	Alfa Belanja	-	Rp17.100	Rp5.023.584
2026-08-04	Mandiri	Jual / Tarik Emas: Emas	Rp315.000	-	Rp5.040.684
2026-08-03	Blu	Grab Transport	-	Rp12.900	Rp4.725.684

Gambar 3.19 Halaman laporan buku besar bulanan

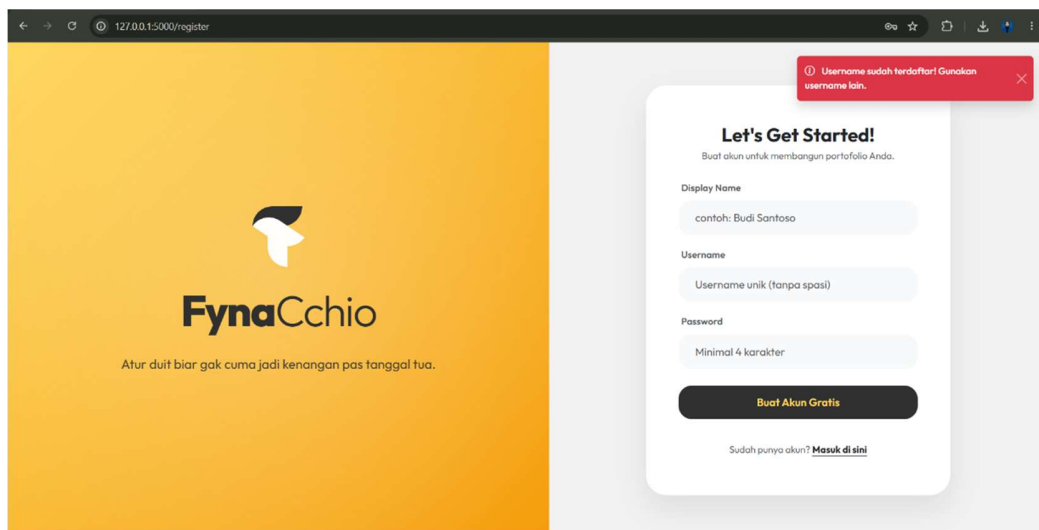


LAPORAN ARUS KAS - 2026-08
Dicetak pada: 2026-08-06 16:13:44 oleh tinn

Tanggal	Akun	Pos	Deskripsi	Masuk (Db)	Keluar (Cr)	Saldo
2026-08-01	Blu	Tagihan	Am	-	Rp8.325	Rp4.793.617
2026-08-01	Blu	Tagihan	FI	-	Rp5.439	Rp4.788.178
2026-08-02	Blu	Tagihan	Axis	-	Rp6.599	Rp4.781.579
2026-08-03	Blu	Freelance	Genshin	Rp15.000	-	Rp4.796.579
2026-08-03	Blu	Tagihan	Grab	-	Rp12.900	Rp4.783.679
2026-08-03	Blu	Belanja	Alfa	-	Rp9.600	Rp4.774.079
2026-08-03	Blu	Makanan	Grab	-	Rp35.485	Rp4.738.584
2026-08-03	Blu	Transport	Grab	-	Rp12.900	Rp4.725.684
2026-08-04	Mandiri	Emas	Jual / Tarik	Rp315.000	-	Rp5.040.684
2026-08-04	Mandiri	Belanja	Alfa	-	Rp17.100	Rp5.023.584
2026-08-04	GoPay	Transport	Grab	-	Rp12.700	Rp5.010.884
TOTAL				Rp330.000	Rp121.058	

Gambar 3.20 Hasil ekspor laporan dalam format PDF

3.2 Penanganan Error (Exception)



Let's Get Started!
Buat akun untuk membangun portofolio Anda.

Display Name
contoh: Budi Santoso

Username
Username unik (tanpa spasi)

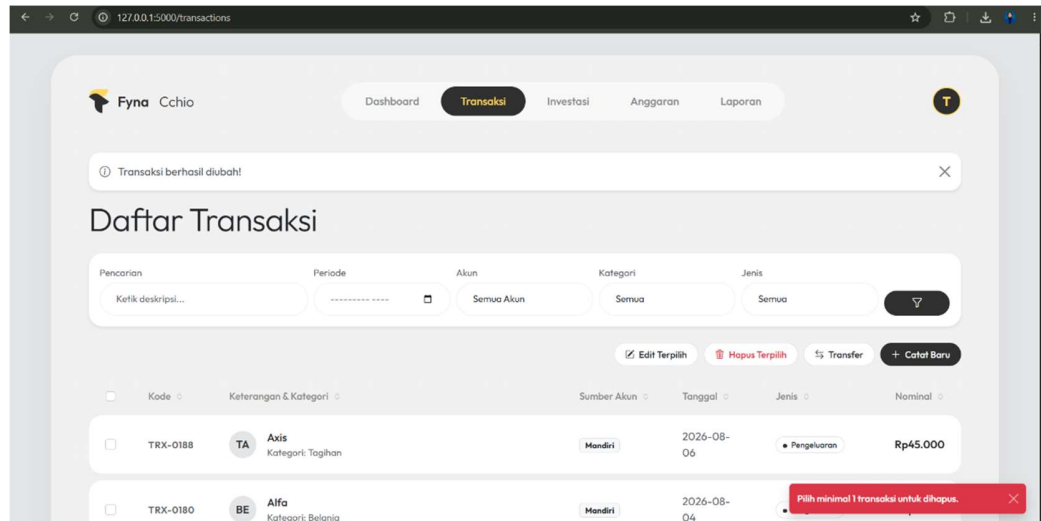
Password
Minimal 4 karakter

Buat Akun Gratis

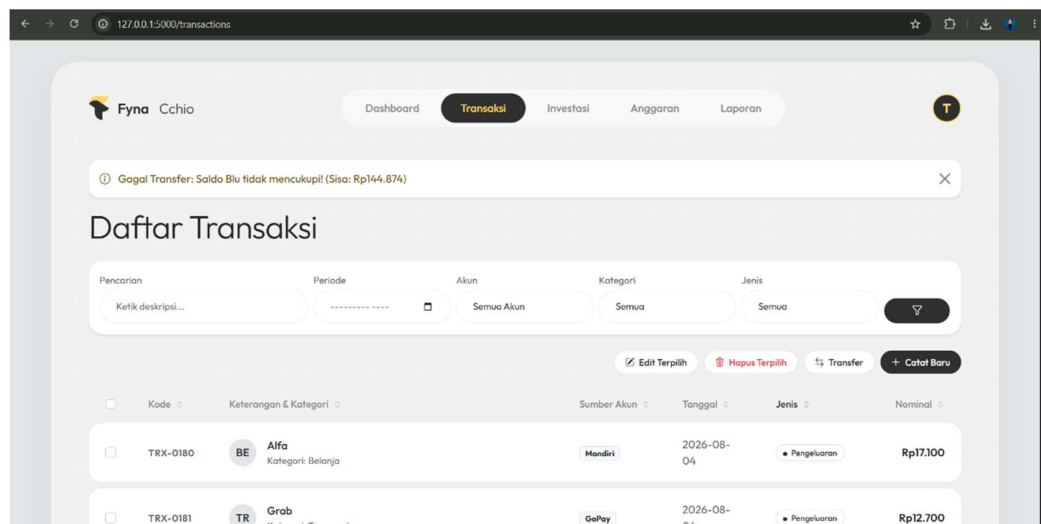
Sudah punya akun? [Masuk di sini](#)

Error Message: Username sudah terdaftar! Gunakan username lain.

Gambar 3.21 Penolakan registrasi karena nama pengguna telah digunakan



Gambar 3.22 Peringatan penghapusan tanpa memilih transaksi



Gambar 3.23 Penolakan transfer karena saldo akun sumber tidak mencukupi

BAB IV

KESIMPULAN

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian, FynaCchio telah berhasil dibangun sebagai aplikasi pengelolaan keuangan pribadi dan investasi berbasis web. Aplikasi ini mampu menyediakan fitur registrasi dan login pengguna, pengelolaan profil, pencatatan pemasukan dan pengeluaran, transfer dana antar akun, pengelolaan portofolio investasi, target tabungan, anggaran, serta penyusunan laporan buku besar dan ekspor laporan ke dalam format PDF.

FynaCchio menggunakan Flask sebagai kerangka kerja aplikasi, MySQL sebagai *database*, serta HTML, CSS, JavaScript, Bootstrap, dan Chart.js untuk membentuk antarmuka dan visualisasi data. Data setiap pengguna dipisahkan menggunakan *user_id* dan akses ke halaman tertentu dilindungi menggunakan *session* serta *login_required()*. Struktur *database* juga dilengkapi dengan *primary key*, *foreign key*, dan batasan unik untuk menjaga hubungan serta konsistensi data.

Hasil pengujian menunjukkan bahwa proses utama aplikasi, terutama operasi *create*, *read*, *update*, dan *delete* pada data transaksi, telah dapat dijalankan melalui antarmuka web. Sistem juga mampu menangani beberapa kesalahan, seperti penggunaan nama pengguna yang telah terdaftar, penghapusan tanpa memilih transaksi, dan transfer dana ketika saldo akun sumber tidak mencukupi. Dengan demikian, FynaCchio telah memenuhi tujuan utama sebagai sistem pencatatan keuangan yang menggabungkan transaksi harian, anggaran, tabungan, investasi, analisis keuangan, dan laporan dalam satu aplikasi.

4.2 Tautan Wajib

- Link Youtube Video Demo : <https://youtu.be/g8pDMGlcUtg?si=Sn0TT6U5B7nUjy90>
- Link GitHub Repository Code : <https://github.com/vexiatitania-ttn/Bahasa-Pemrograman>